

Chap-15 Tensor Processing Unit (TPU)

1. Introduction to TPU

TPU (Tensor Processing Unit) is a **specialized chip developed by Google**. It is designed for **ultra-fast execution of neural network operations**. It focuses only on **matrix and tensor computations**. It acts as a **co-processor for deep learning tasks**.

2. C/G/T Processing Units

- **CPU (Central Processing Unit):**
 - General-purpose processor.
 - Handles all types of operations (logic, arithmetic, memory).
 - Based on **Moore's Law**.
- **GPU (Graphics Processing Unit):**
 - Designed for **graphics and parallel processing**.
 - Performs operations like:
 - Matrix multiplication
 - Rasterization
 - Texture mapping
 - Later found useful in **deep learning (serendipity)**.
- **TPU (Tensor Processing Unit):**
 - Designed **only for deep learning computations**.
 - Removes unnecessary graphics operations.
 - More **efficient and faster** than GPUs for ML tasks.

3. Limitations of GPUs

- GPUs include **graphics-specific hardware** not needed for AI.
- Wasted resources for:
 - Rasterization
 - Frame buffer operations
 - Texture mapping
- Not fully optimized for **tensor operations**.

4. Origin of TPU (TPU v1)

- It is proposed by **Jeff Dean** in 2013.
- The increasing user demand required **more servers**. So, TPU was designed.
- Its major goal is to achieve **10× performance improvement without cost increase**.
- TPU v1 was developed in just **15 months**.

Tensor Processing Unit are **specialized hardware accelerators** designed by Google specifically for **machine learning and deep learning workloads**.

They are optimized for **tensor operations** (matrix multiplications, vector operations), which are the core of neural networks.

TPUs are Needed

Deep learning involves heavy computations like:

- Matrix multiplication (e.g., in CNNs, Transformers)
- Backpropagation
- Large-scale parallel operations

Traditional processors:

- CPU → general purpose (slow for DL)
- GPU → parallel but still general

TPUs are **custom-built only for tensor math**, so they are much faster.

5. TPU Architecture

Key components:

1. Matrix Multiply Unit (MXU)

- Core of TPU
- Performs large matrix multiplications
- Uses **systolic arrays**

2. High Bandwidth Memory (HBM)

- Stores tensors efficiently
- Fast data access

3. Vector Processing Unit

- Handles activation functions (ReLU, sigmoid)

- TPU is an **ASIC (Application-Specific Integrated Circuit)**. It is based on “less is more” philosophy.
- Eliminates unnecessary hardware and improves speed and energy efficiency

Name	LOC	Layers					Nonlinear Function	Weights	TPUv1 Ops / Weight Byte	TPUv1 Batch Size	% Deployed
		FC	Conv	Vector	Pool	Total					
MLP0	0.1k	5				5	ReLU	20M	200	200	61%
MLP1	1k	4				4	ReLU	5M	168	168	
LSTM0	1k	24		34		58	sigmoid, tanh	52M	64	64	29%
LSTM1	1.5k	37		19		56	sigmoid, tanh	34M	96	96	
CNN0	1k		16			16	ReLU	8M	2888	8	5%
CNN1	1k	4	72		13	89	ReLU	100M	1750	32	

Figure 15.1: Different inference workloads seen at Google in 2013 (source [3])

This figure concludes that: Different models have different structure, computation needs, memory usage. This diversity motivated the design of **TPUs optimized for tensor operations**.

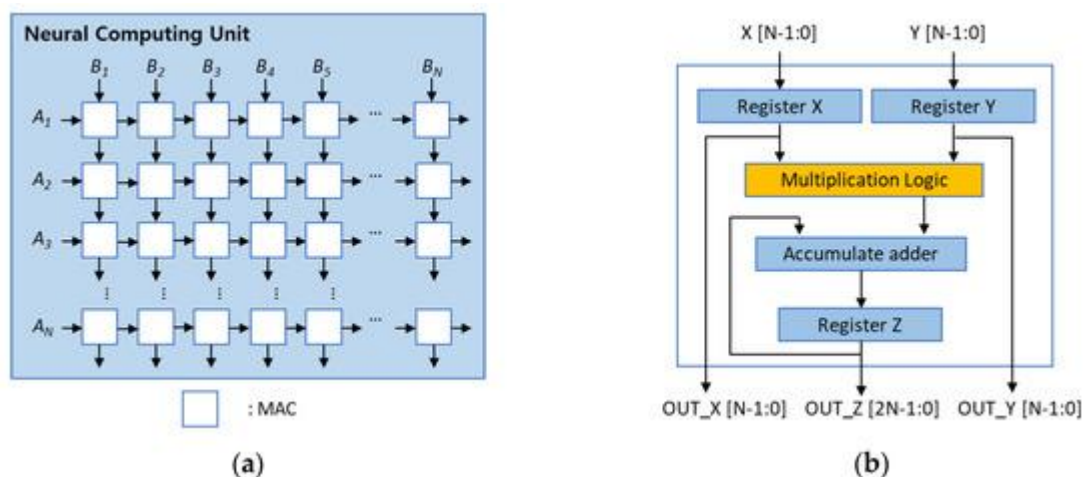
The TPU is designed to **accelerate matrix multiplication for neural networks** using a **systolic array**.

Systolic Array

A **systolic array** is a grid of simple processing units (PEs) where data flows **rhythmically (like a heartbeat)** and each unit performs (**Multiply + Add (MAC)**) and the data moves from one unit to another in a **pipeline**

It is basically used for Matrix multiplication, CNNs and Deep learning accelerators (TPUs).

The structure is basically



Each **PE (Processing Element)**:

- Receives input from left & top
- Multiplies values
- Adds to partial sum
- Passes data to right & bottom

Matrix A flows horizontally and matrix B flows vertically and each PE computes:

$\text{partial_sum} += A \times B$ and final result is collected.

TPU uses 256X256 systolic arrays and perform $256 \times 256 = 65,536$ MAC operations per cycle

Basic Concept of TPUs

- TPUs are **domain-specific processors** developed by Google.
- Optimized for **matrix (tensor) operations** used in deep learning.
- Core operation: **Matrix multiplication (dot product)**.

What is Matrix Multiplication and what is the problem in it?

- Each output element is computed as:
 - **Row of one matrix $\times$ Column of another matrix**
- Sequential computation:
 - Time complexity = **$O(n^3)$**
- Problem:
 - Very slow for large matrices $\rightarrow$ not suitable for deep learning.

What is the need for TPUs?

- CPUs $\rightarrow$ General purpose (slow for ML)
- GPUs $\rightarrow$ Include unnecessary graphics hardware
- TPUs $\rightarrow$ Designed **only for tensor operations**
- TPU uses **systolic arrays** for computation which is called Systolic Matrix Multiplication:
 - In. Systolic Matrix Multiplication, data flows in a **rhythmic, pipeline manner**, inspired by **heartbeat (systolic flow)**. It performs **parallel computation**.

1. First Generation Features of TPU v1 (Announced in 2016 at Google I/O)

- Uses **8-bit arithmetic**
- Designed for **inference only**
- Does **not support training** (needs floating point)
- Acts as a **co-processor controlled by CPU**

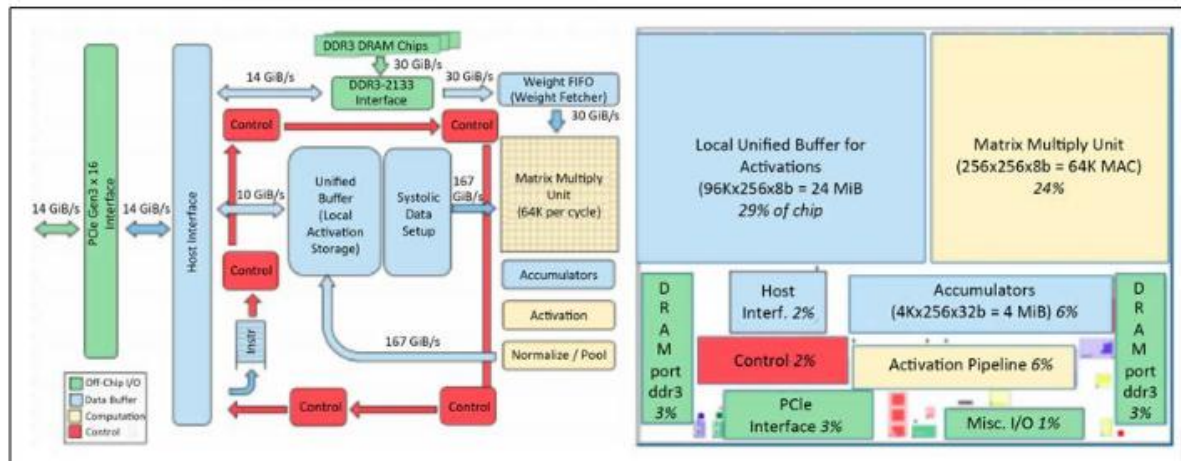


Figure 15.2: TPU v1 design schema (source [3])

From the above diagram:

PCIe Interface is data input path that connects TPU to CPU and the bandwidth is 14GB/s and transfer input data model weights and instruction.

Host Interface:

It controls the communication between CPU and TPU and send command like start computation and load weight.

DDR3 DRAM: It store model weights and input data.

Unified Buffer: size is 24 MB stores activations (immediate output). Objective is to reduce slow DRAM access, enables faster reuse of data.

Weight FIFO (Handles weight) It fetches weight from DRAM and streams them into compute unit. Ensures continuous data flow

Matrix Multiply Unit (MXU) Core computation unit. It perform 65536 MAC operation per cycle.

Here inputs (activations) flow horizontally and weights flow vertically and multiply + accumulate happens at each cell.

So TPU are faster.

Accumulators (4 MB)

It Store intermediate sums from MXU and prevent precision loss. It is important for back propagation and large matrix operations.

Activation Pipeline

It is Activation + Normalize + Pool applies ReLU / Sigmoid, Batch normalization and pooling. This completes one neural network layer.

Control Unit

It manages data movement and execution flow

Basically:

- CPU sends data via PCIe
- Data stored in DDR3 memory
- Weights fetched via FIFO
- Activations stored in Unified Buffer
- MXU performs matrix multiplication
- Results stored in Accumulators
- Activation functions applied
- Output sent back or reused

TPU v1 Architecture

- Components:
 - **MMU (Matrix Multiply Unit)** → performs multiplications. It contains 256×256 systolic arrays. Total operations in parallel: 65,536 multiplications at once
 - **Unified buffer** → stores intermediate data
 - **Activation unit** → applies functions (ReLU, etc.)
- Memory:
 - Uses **DDR4 DRAM** for weights storage
- Communication:
 - CPU ↔ TPU via **PCIe 3.0 bus**

TPU v1 Working

1. CPU sends instructions to TPU
2. TPU fetches weights from memory
3. MMU performs **systolic multiplication**
4. Activation functions are applied
5. Output is returned

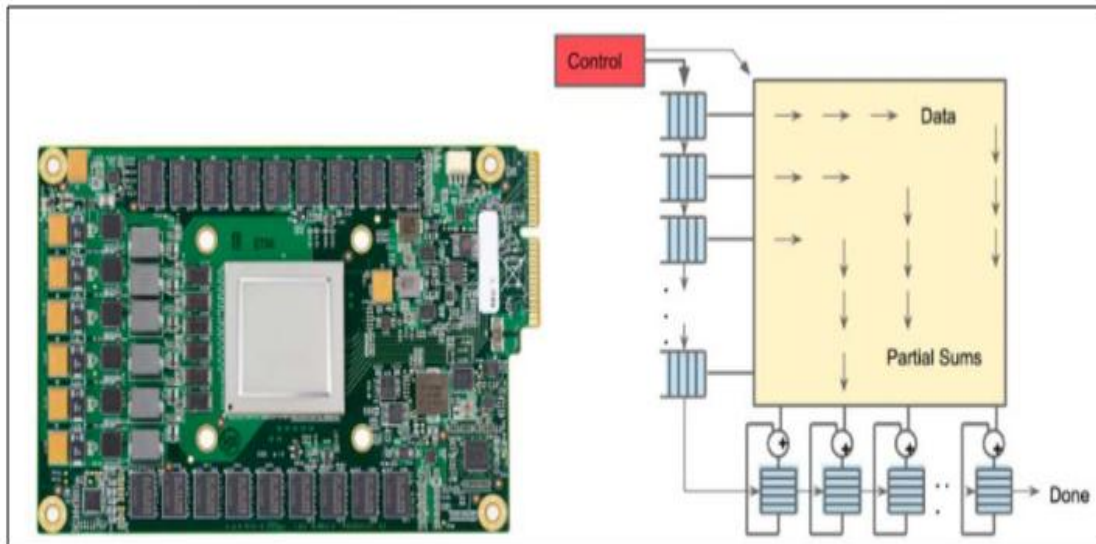


Figure 15.3: On the left you can see a TPU v1 board, and on the right an example of how the data is processed during the systolic computation

2. Second Generation TPU (TPU v2) – 2017

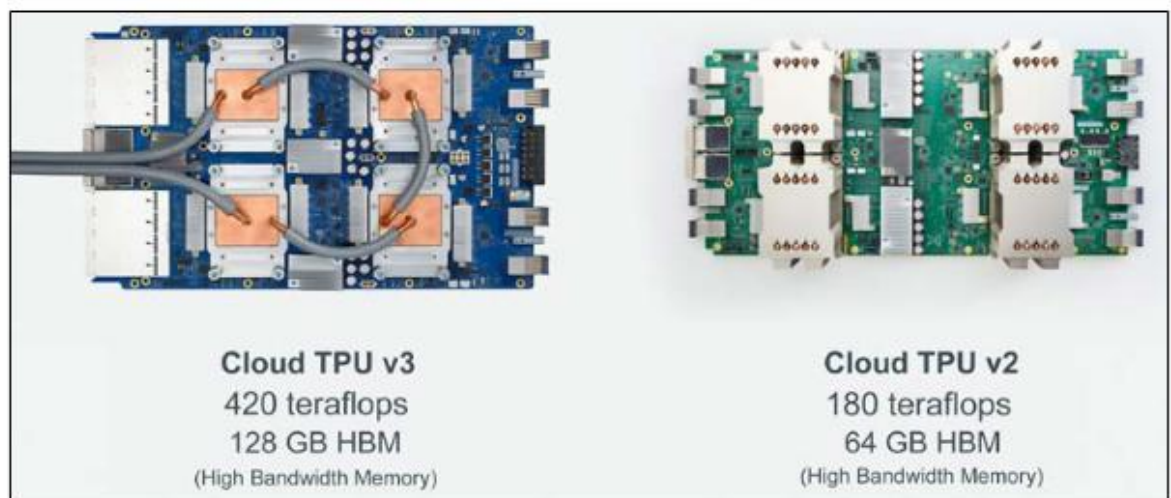


Figure 15.5: Cloud TPU v2 and Cloud TPU v3

- Developed by Google.
- Performance:
 - **45 TFLOPS per TPU**
 - **TFLOPS = Tera Floating Point Operations Per Second**
 - **1 TFLOP = 1 trillion calculations per second**
 - **So: 1 TPU = 45 trillion operations per second**
 - **180 TFLOPS per module (4 TPUs)**
 - A TPU module contains 4 TPU chips
 - So: $45 \text{ TFLOPS} \times 4 = 180 \text{ TFLOPS}$
 - Means, One module can do 180 trillion operations per second

- **11.5 PFLOPS per pod (64 modules)**
 - $180 \text{ TFLOPS} \times 64 = 11,520 \text{ TFLOPS}$
 - $1 \text{ PFLOP} = 1000 \text{ TFLOPS}$
 - So: $11,520 \text{ TFLOPS} = 11.5 \text{ PFLOPS}$
 - Means: A full TPU pod can do **11.5 quadrillion operations per second**

Key Features

- Supports **both training and inference** (unlike TPU v1).
- Uses **floating-point arithmetic**.
- Memory bandwidth increased to **600 GB/s**.
- Benefits:
 - Faster computation
 - Lower power consumption
 - Efficient for deep learning

3. Third Generation TPU (TPU v3) – 2018

- **2× faster than TPU v2**
- Pods are **4× larger**
- Total improvement:
 - **8× performance increase**

Performance

- TPU v3 Pods:
 - **100+ PFLOPS**

Architecture

- 1 board:
 - 4 chips
 - 8 cores
- Uses **liquid cooling**

Advanced Features

- High-speed interconnect (supercomputer-level)
- Uses **reduce-all algorithm**:
 - Synchronizes all TPUs during training
- It works like a **massive supercomputer for AI**

4. Fourth Generation TPU (TPU v4)

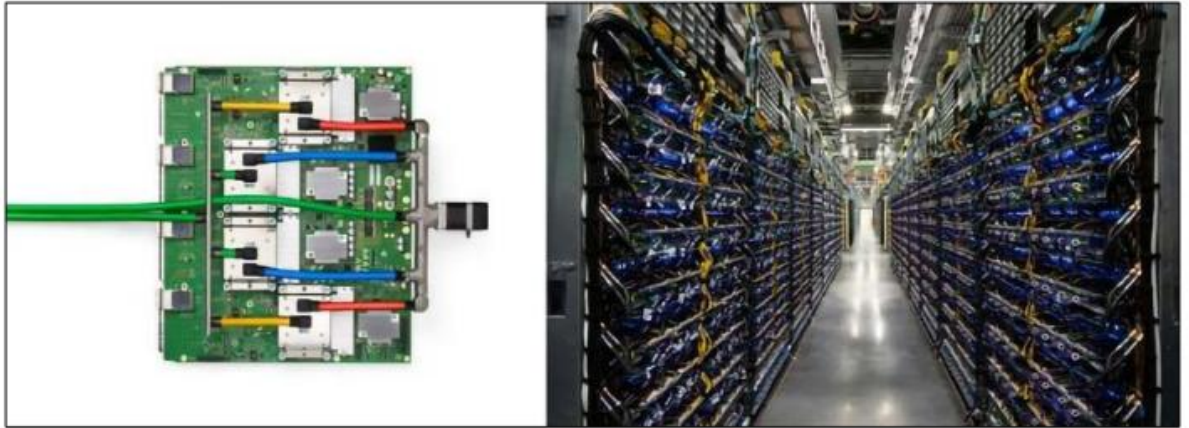


Figure 15.7: A TPU v4 chip and a portion of a TPU v4 Pod – source: <https://twitter.com/google/>

- Latest advanced TPU by Google.

Performance

- **275 TFLOPS per chip**
- **1.1 exaflops per pod**

Improvements

- 2× more powerful than TPU v3
- Better:
 - Memory bandwidth
 - Interconnect technology

Usage

- Used in advanced AI systems like:
 - Search
 - Assistant
 - Translate
- Supports large models (billions/trillions of parameters)

5.Edge TPU (2018)

- Designed for **edge devices (IoT & mobile)**.

Performance

- **4 TOPS (trillion operations/sec)**
- Power consumption: **~2 watts**

Key Features

- Small, low-power, efficient
- Supports **TensorFlow Lite**
- Ideal for:
 - Mobile devices
 - Embedded systems
 - Real-time ML

Example

- Used in **Pixel 4 smartphone (Pixel Neural Core)**

Accessing TPUs

- Available via:
 - Google Cloud (GCE)
 - Kubernetes Engine (GKE)
 - **Google Colab (free access)**

Difference between TPU v2, TPU v3, TPU v4 and Edge TPU

- **TPU v2:** Training + inference, bfloat16, 11.5 PFLOPS pods
- **TPU v3:** Faster, scalable, supercomputer-level performance
- **TPU v4:** Exaflop-level performance, used in advanced AI
- **Edge TPU:** Low-power, on-device ML for IoT

TPU Performance

- TPU performance depends on:
 - **Metrics used** (e.g., FLOPS, images/sec)
 - **Workloads/benchmarks** (e.g., ResNet-50)
- Common benchmark:
 - Deep learning models like **ResNet-50**
- **TPU v2 vs GPU**
 - A full **Cloud TPU v2 Pod** is:
200× faster than an NVIDIA Tesla V100
 - Especially for training deep learning models

- **TPU v4 Performance**
 - Achieves top results in MLPerf 1.0
 - Competes strongly with NVIDIA A100 GPUs

2. Linear Scalability

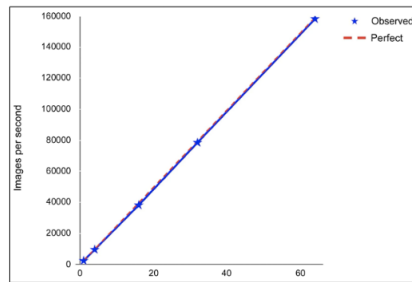


Figure 15.9: Linear scalability in the number of TPUs v2 when increasing the number of Images. X-axis: Number of TPUs / workload size, Y-axis: Images processed per second, Two lines: **Observed (actual performance)**, **Perfect (ideal scaling)**

- Shows **TPU v2 scaling with workload size**.
 - As the number of TPUs increases: **Performance increases linearly**

3. What is Linear Scaling?

- If TPUs double → performance also doubles
 - Means: Efficient **parallel processing**, Minimal overhead

5. TPU v2 Pod Performance

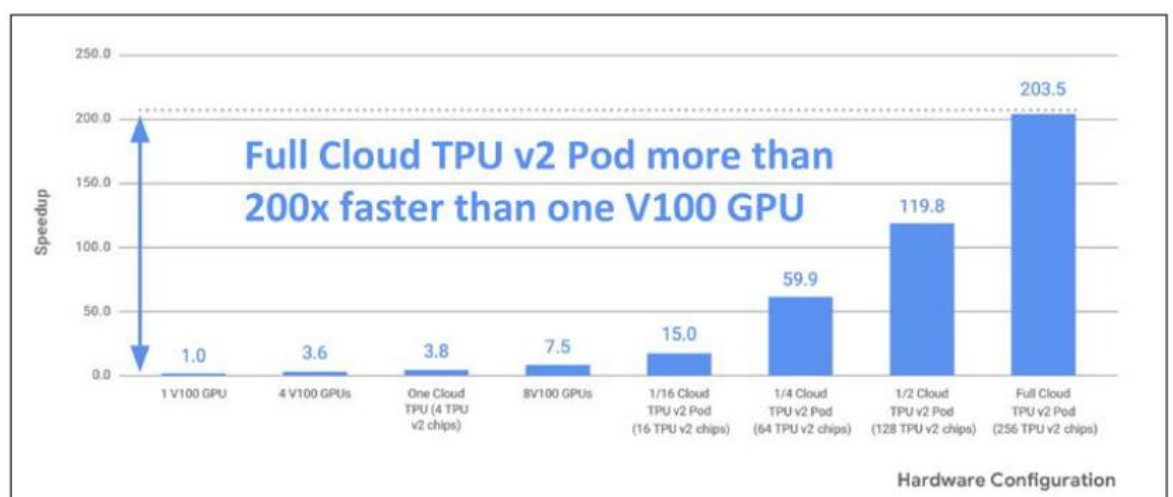


Figure 15.10: A full Cloud TPU v2 Pod is >200x faster than a V100 NVIDIA Tesla GPU for training a ResNet-50 model

- Comparison with GPU (NVIDIA V100)

- **Full Cloud TPU v2 Pod is >200× faster than a V100 GPU**

Because:

- TPU Pod:
 - Many TPUs working together
 - Massive parallelism
- GPU:
 - Limited compared to TPU clusters

5. Performance Comparison

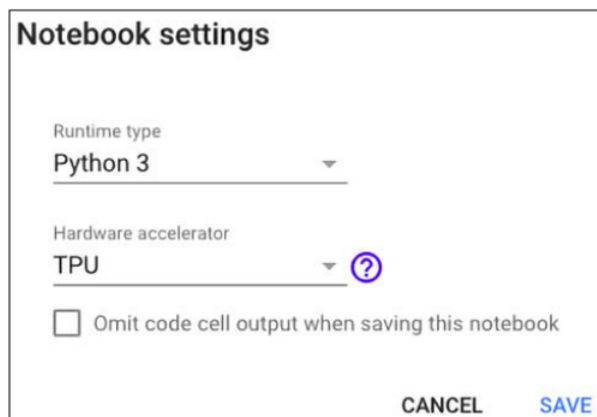
Hardware	Performance
CPU	Lowest
GPU (V100)	High
TPU v2 Pod	Extremely High (200× GPU)

What is the Reasons for High TPU Performance?

- Optimized for **matrix operations**
- Uses **systolic arrays**
- High **memory bandwidth**
- Efficient **parallelism across pods**
- Specialized for **deep learning workloads**

How to use TPUs with Colab

In this section, we show how to use TPUs with Colab. Just point your browser to <https://colab.research.google.com/> and change the runtime from the Runtime menu as shown in Figure 15.12. First, you'll need to enable TPUs for the notebook, then navigate to Edit→Notebook settings and select TPU from the Hardware accelerator drop-down box:



Checking whether TPUs are available

First of all, let's check if there is a TPU available, by using this simple code fragment that returns the IP address assigned to the TPU. Communication between the CPU and TPU happens via gRPC (gRPC Remote Procedure Call), which is a modern, open-source, high-performance Remote Procedure Call (RPC) framework that can run in any environment:

Step 1: Import TensorFlow

```
import tensorflow as tf
```

Print TensorFlow version

```
print("TensorFlow version:", tf.__version__)
```

Step 2: Detect TPU

```
try:
```

```
    # Try to detect TPU (only works if TPU runtime is enabled)
```

```
    tpu = tf.distribute.cluster_resolver.TPUClusterResolver()
```

```
    print("TPU detected")
```

```
    print("TPU workers:", tpu.cluster_spec().as_dict()['worker'])
```

```
except ValueError:
```

```
    # If TPU is not found, raise an error
```

```
    raise BaseException(
```

```
)
```

Step 3: Connect to TPU cluster

Connect TensorFlow to the TPU cluster

```
tf.config.experimental_connect_to_cluster(tpu)
```

Step 4: Initialize TPU system

Initialize the TPU hardware

```
tf.tpu.experimental.initialize_tpu_system(tpu)
```

```
print("TPU initialized successfully")
```

Step 5: Create TPU strategy

This strategy allows distributed training on TPU

```
tpu_strategy = tf.distribute.experimental.TPUStrategy(tpu)
```

```
print("TPU Strategy created")
```

```

Tensorflow version 2.8.0
Running on TPU ['10.36.66.50:8470']
INFO:tensorflow:Deallocate tpu buffers before initializing tpu system.
INFO:tensorflow:Deallocate tpu buffers before initializing tpu system.
INFO:tensorflow:Initializing the TPU system: grpc://10.36.66.50:8470
INFO:tensorflow:Initializing the TPU system: grpc://10.36.66.50:8470
INFO:tensorflow:Finished initializing TPU system.
INFO:tensorflow:Finished initializing TPU system.
WARNING:absl:'tf.distribute.experimental.TPUStrategy' is deprecated, please use
the non experimental symbol 'tf.distribute.TPUStrategy' instead.
INFO:tensorflow:Found TPU system:
INFO:tensorflow:Found TPU system:
INFO:tensorflow:*** Num TPU Cores: 8
INFO:tensorflow:*** Num TPU Cores: 8
INFO:tensorflow:*** Num TPU Workers: 1
INFO:tensorflow:*** Num TPU Workers: 1
INFO:tensorflow:*** Num TPU Cores Per Worker: 8
INFO:tensorflow:*** Num TPU Cores Per Worker: 8

```

- **TensorFlow version** → Confirms installation
- **TPU detected** → Your runtime is correctly set to TPU
- **TPU workers** → Internal TPU network address
- **Initializing TPU system** → Hardware setup
- **Num TPU Cores: 8** → TPU has 8 cores (parallel processing)
- **TPU Strategy created** → Ready for distributed training

1. Keras MNIST TPU End-to-End Training

STEP : Detect TPU / GPU / CPU

=====

try:

Try TPU

tpu = tf.distribute.cluster_resolver.TPUClusterResolver.connect()

strategy = tf.distribute.TPUStrategy(tpu)

print("Running on TPU")

except ValueError:

Fallback to GPU or CPU

strategy = tf.distribute.MirroredStrategy()

print("Running on GPU/CPU")

```
print("Number of accelerators:", strategy.num_replicas_in_sync)
```

Output:

Case 1: TPU is ENABLED (Best case)

Running on TPU

Number of accelerators: 8

Meaning:

- TPU is successfully detected
- 8 = number of TPU cores (parallel processing)

Case 2: GPU is ENABLED

Running on GPU/CPU

Number of accelerators: 1

Meaning:

- TPU not found → fallback to GPU
- Usually 1 GPU is used

Case 3: Only CPU

Running on GPU/CPU

Number of accelerators: 1

Meaning:

- No TPU, no GPU → CPU is used
- Still shows 1 (single device)

This code uses **TensorFlow distribution strategies** to automatically select the best hardware:

- **TPU** → fastest (if available)
- **GPU (via MirroredStrategy)** → fallback
- **CPU** → last option

What are Pretrained TPU Models?

- Google provides **ready-made deep learning models** trained on TPUs in the tensorflow/tpu GitHub repository

- These models are already trained on **large datasets**, so you don't need to train from scratch.

Why use Pretrained Models?

- Saves **time and computation**
- Gives **better accuracy**
- Enables **Transfer Learning**
- Optimized for TPU performance

Types of Models Available

1. Image Recognition & Computer Vision

- ResNet-50 / 101 / 152
- Inception v3 / v2 / v4
- AmoebaNet

Used for:

- Image classification
- Feature extraction

2. Object Detection & Segmentation

- Mask R-CNN
- RetinaNet
- DeepLab

Used for:

- Detecting objects
- Image segmentation

3. Lightweight / Mobile Models

- MobileNet
- MnasNet
- SqueezeNet

Used for:

- Low-resource devices
- Mobile applications

4. NLP (Language Models)

- BERT
- Transformer-based models

Used for:

- Machine translation
- Sentiment analysis
- Question answering

5. Speech Recognition

- Transformer-based ASR models

Converts:

- Speech → Text

6. Image Generation

- DCGAN
- GANs
- Image Transformer

Used for:

- Generating new images

How to Use Them

Step 1: Clone repository

```
git clone https://github.com/tensorflow/tpu
```

Step 2: Load pretrained model

Example:

```
model = tf.keras.applications.ResNet50(weights='imagenet')
```

Step 3: Fine-tune

```
for layer in model.layers[:-10]:  
    layer.trainable = False
```

Step 4: Train on your dataset

```
model.fit(...)
```

Full code of Keras MNIST TPU End-to-End Training (For lab)

```
# =====
# STEP 1: Import libraries
# =====
import tensorflow as tf
from tensorflow import keras

print("TensorFlow version:", tf.__version__)

# =====
# STEP 2: Detect TPU / GPU / CPU
# =====

try:
    # Try TPU
    tpu = tf.distribute.cluster_resolver.TPUClusterResolver.connect()
    strategy = tf.distribute.TPUStrategy(tpu)
    print("Running on TPU")

except ValueError:
    # Fallback to GPU or CPU
    strategy = tf.distribute.MirroredStrategy()
    print("⚠ Running on GPU/CPU")

print("Number of accelerators:", strategy.num_replicas_in_sync)

# =====
# STEP 3: Load MNIST dataset
# =====

(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()

# Normalize data (0-255 → 0-1)
```

```
x_train = x_train / 255.0
```

```
x_test = x_test / 255.0
```

```
# Flatten images (28x28 → 784)
```

```
x_train = x_train.reshape(-1, 784)
```

```
x_test = x_test.reshape(-1, 784)
```

```
# =====
```

```
# STEP 4: Build model inside strategy scope
```

```
# =====
```

```
with strategy.scope():
```

```
    model = keras.Sequential([
        keras.layers.Dense(128, activation='relu', input_shape=(784,)),
        keras.layers.Dense(64, activation='relu'),
        keras.layers.Dense(10, activation='softmax')
    ])
```

```
    model.compile(
        optimizer='adam',
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy']
    )
```

```
print(" Model built successfully")
```

```
# =====
```

```
# STEP 5: Train model
```

```
# =====
```

```
model.fit(
    x_train, y_train,
    epochs=5,
    batch_size=128
)
```

```
# =====
```

```
# STEP 6: Evaluate model
```

```
# =====
```

```
loss, acc = model.evaluate(x_test, y_test)
```

```
print(f"\nTest Accuracy: {acc * 100:.2f}%")
```